



Lecture 11: Callback Props

INTERVIEW QUESTION

[> CORE](#)

How does a child component tell its parent that something happened?

Your `Button` is clicked, but the parent needs to know. How does the child send a message up the tree without a global store?

Data in Svelte flows strictly downwards via props. But applications are interactive; when a user takes an action inside a nested component, the broader application needs to react.

The Upward Communication Problem

Imagine you are building an e-commerce site. You have an `App.svelte` parent component that holds the shopping cart state. Inside it, you render a list of `<ProductCard>` child components, each featuring an "Add to Cart" button.

When a user clicks that button, the click event fires completely inside the child component. However, the shopping cart data lives up in the parent. If the child has no way to alert the parent, the user clicks the button repeatedly, the cart remains empty, and the user leaves the site in frustration. You need a reliable way for the child to shout "Hey, this item was clicked!" up the component tree.

Functions as Props

In Svelte 5, the solution is incredibly simple: functions are just JavaScript values, which means you can pass them as props.

The parent component defines a function that updates its own state. It then passes that exact function down to the child component as a prop (often called a **callback**).

● ● ● App.svelte

```
1  <script>
2    import ProductCard from './ProductCard.svelte';
3
4    let cartCount = $state(0);
5
6    function handleAdd() {
7      cartCount++;
8    }
9  </script>
10
11  <p>Items in cart: {cartCount}</p>
12
13  <ProductCard onadd={handleAdd} />
```

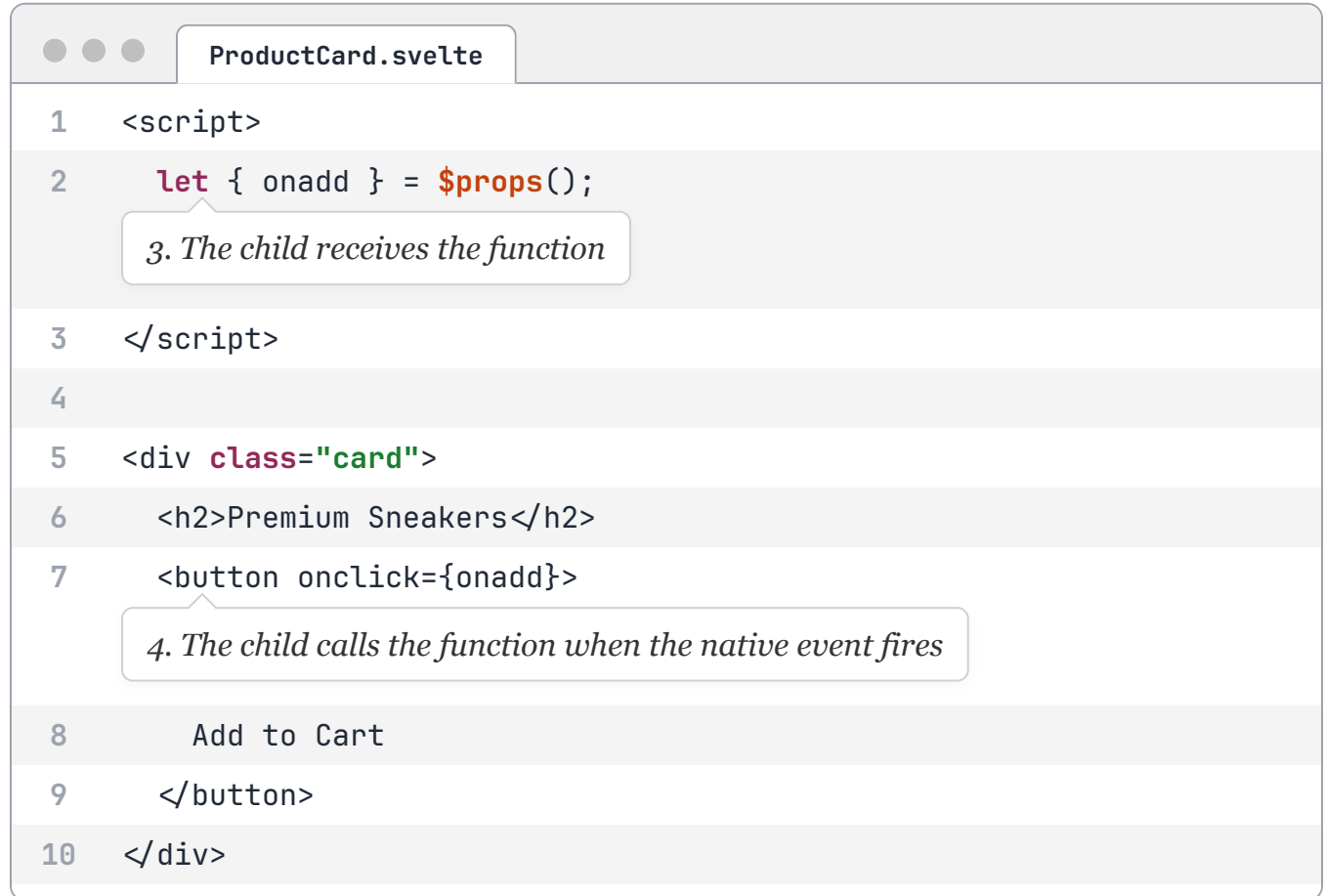
1. The parent defines the function

2. The parent passes the function down as a prop

Triggering the Callback

Inside the child component, it declares the callback function in its `$props()` block just like any other data. When the user interacts with the UI, the child simply executes the function.

Because the function was defined in the parent's scope, executing it updates the parent's state.



```
1  <script>
2    let { onadd } = $props();
3  </script>
4
5  <div class="card">
6    <h2>Premium Sneakers</h2>
7    <button onclick={onadd}>
8      Add to Cart
9    </button>
10  </div>
```

3. The child receives the function

4. The child calls the function when the native event fires

Passing Data Upwards

Callback functions aren't just for signaling that an event happened; they can also carry data. If the child needs to tell the parent exactly *what* was clicked, it can pass arguments into the callback function.

● ● ●

ProductCard.svelte

```
1  <script>
2    let { id, onadd } = $props();
3  </script>
4
5  <button onclick={() => onadd(id)}>
    The child passes its own data back up to the parent
6    Add to Cart
7  </button>
```

The parent function `function handleAdd(productId)` will receive the ID directly from the child.

INTERVIEW TIP

To impress the interviewer: If they ask about `createEventDispatcher`, confidently explain that it was deprecated in Svelte 5. Callback props have replaced custom event dispatchers because they are simpler, faster, and provide vastly superior type-safety natively in TypeScript without requiring custom generic types.

Summary

While data flows down, events flow up. To allow a child component to communicate with its parent in Svelte 5, the parent passes a function down as a prop. The child executes this callback function when an action occurs, optionally passing internal data as arguments. Because the function is bound to the parent's scope, it can safely modify the parent's state, bridging the gap between nested components seamlessly.

	SVELTE 4 (Legacy)	SVELTE 5 (Modern)
Mechanism	<code>createEventDispatcher()</code>	Callback functions via <code>\$props()</code>
Parent Syntax	<code><Component on:click={fn} /></code>	<code><Component onclick={fn} /></code>
Type Safety	Complex custom generic interfaces	Native TypeScript function types